

scitech-librarian — Manual do Usuário

versão 3.5.1

2026-09-04

Sumário

1. O que é	2
2. Instalação e configuração	2
3. Conceitos	3
4. Escrevendo consultas	3
5. Executando uma busca	4
6. O diretório de pesquisa	5
6.1 Índice	5
6.2 Trazendo registros de fora	5
6.3 Do Zotero, Mendeley e EndNote	6
7. Relatórios	6
7.1 Uma rodada	6
7.2 O diretório de pesquisa inteiro	6
7.3 Níveis	6
7.4 Formatos	7
7.5 Filtros	7
7.6 PRISMA	7
7.7 Sugestões	8
7.8 Idiomas	8
8. Métricas de periódicos	8
9. Web of Science	9
10. Logs e auditoria	9
11. Fluxos de trabalho	9
12. Recursos e limitações	10
13. Testes	10
14. Licença e conduta	10

[English](#) · **Português (Brasil)** · [Español](#) · [Deutsch](#) · [Français](#)

Tradução do manual em inglês, que é a referência; comandos, nomes de arquivos, opções e blocos de código ficam como no original.

1. O que é

O scitech-librarian é um instrumento reprodutível de busca bibliográfica para ciência e engenharia. Você escreve uma consulta estruturada uma vez; ele a executa em até nove bases bibliográficas por meio de suas APIs documentadas, arquiva tudo (registros, a string de consulta exata enviada a cada base, contagens de resultados, um log) e escreve um relatório de busca bibliográfica com um diagrama de fluxo PRISMA 2020. Ao longo de meses, as rodadas, mais os registros obtidos por outros meios, se acumulam em um **diretório de pesquisa** que o mesmo relatório pode descrever como um todo — o que cada busca acrescentou, o que cada base contribuiu, como as contagens derivaram, quais veículos importam.

São cinco scripts Python mais dois módulos compartilhados (`render.py`, `i18n.py`), sem dependências além da biblioteca padrão. Não há nada a instalar: copie os arquivos, forneça suas chaves, escreva `queries.json`, rode.

Arquivo	Papel
<code>librarian.py</code>	executa uma busca; arquiva uma rodada; chama o relatório
<code>project.py</code>	diretório de pesquisa: índice, ingestão de registros externos, status
<code>report.py</code>	relatórios de uma rodada ou do diretório inteiro; PRISMA; filtros
<code>journals.py</code>	métricas de periódicos (números do tipo fator de impacto) por ano
<code>wos_manual.py</code>	Web of Science à mão (sem API gratuita utilizável)
<code>render.py</code>	renderizadores Markdown / HTML / LaTeX / texto e a cadeia de PDF (importado por <code>report.py</code>)
<code>i18n.py</code>	idiomas do relatório: o catálogo em / pt-BR / es / de / fr (importado por <code>report.py</code> ; §7.8)

Para agentes de IA. `AGENTS.md` na raiz do repositório é a descrição completa da ferramenta orientada à máquina. Se você trabalha com um agente de programação (Claude Code, Codex, Cursor...), diga a ele: “*Leia o AGENTS.md e depois faça uma verificação de novidade sobre X*” — ele contém os comandos, os esquemas de arquivos, os fluxos de trabalho e as regras que o agente não deve quebrar.

2. Instalação e configuração

Requisitos: Python 3.9 ou mais novo. Opcional, para relatórios em PDF tipografados: uma distribuição LaTeX (xelatex, lualatex ou pdflatex) ou o pandoc; sem eles o PDF é produzido por um escritor de texto puro embutido.

```
git clone https://github.com/fabiocampolim-design/scitech-librarian
cd scitech-librarian
cp .env.example .env # fill in what you have (or use the environment)
cp queries.example.json queries.json
python librarian.py --selftest
```

As chaves são lidas do ambiente do processo. Copie `.env.example` para `.env` e preencha esse arquivo, ou defina os mesmos nomes de variável no seu shell, na configuração do seu agente ou lançador, ou como segredos de CI — o `.env` só fornece o que o ambiente ainda não tiver definido, então uma variável definida fora sempre vence e o arquivo é opcional.

Chaves:

Chave	Necessária para	Como obter
<code>CONTACT_EMAIL</code>	acesso ao “polite pool” do OpenAlex/Crossref/Unpaywall	seu endereço

Chave	Necessária para	Como obter
ADS_TOKEN	NASA ADS	gratuito, https://ui.adsabs.harvard.edu/user/settings/token
SCOPUS_API_KEY	Scopus (+ rede institucional/VPN)	gratuita, https://dev.elsevier.com/apikey/manage
SCOPUS_INSTTOKEN	Scopus sem VPN	peça à sua biblioteca
S2_API_KEY	Semantic Scholar mais rápido	opcional
CORE_API_KEY	CORE — opcional; chamadas anônimas funcionam, mas com limite de taxa	gratuita, https://core.ac.uk/services/api
WOS_STARTER_KEY	Web of Science Starter API (gramática restrita)	raramente vale a pena

Seis backends (OpenAlex, arXiv, INSPIRE-HEP, Semantic Scholar, Crossref, CORE) não precisam de chave nem de instituição. `python librarian.py --list` informa quais chaves foram encontradas por qualquer um dos caminhos; `--selftest` prova que elas funcionam.

Uso embutido em outro projeto. Coloque os sete arquivos em um subdiretório `tools/`; `.env`, `queries.json` e `lit/` são então procurados no diretório pai.

3. Conceitos

Bloco. Uma consulta estruturada: uma lista de grupos de sinônimos combinados com AND, cada grupo uma lista de sinônimos combinados com OR. Um bloco tem um nome (`A`, `CD`, `NOV...`), um título e uma nota. Os blocos vivem em `queries.json`.

Rodada. Uma execução de `librarian.py`: cada bloco selecionado contra cada backend selecionado, arquivada em `lit/runs/<timestamp>/`.

Diretório de pesquisa. Uma pasta (padrão `lit/`, escolha outra com `--outdir`) que contém todas as rodadas de um projeto, os registros ingeridos de fora, o índice do projeto (`project.json`), os números da triagem PRISMA (`screening.json`), métricas de periódicos, relatórios e logs de auditoria. Um diretório por projeto; um laboratório tem vários.

Fonte manual. Registros que não vieram de uma rodada: uma exportação do Zotero ou do Mendeley, o arquivo RIS de um colega, uma sessão da Web of Science, uma lista de referências. Ingeridos com `project.py ingest`, mantêm sua proveniência (quem, quando, de onde, método) e aparecem em cada relatório como mais uma fonte, e no fluxo PRISMA na coluna certa.

Registro. O esquema comum que todo arquivo usa: `title year doi journal authors url abstract cited_by issn block backend`. Registros de projeto mesclados também carregam `found_by` (quais fontes o encontraram) e `first_seen`.

Nível. Quanto um relatório contém: `simple` (algumas páginas), `intermediate` (cada registro único mais análises), `full` (tudo, resumos incluídos — centenas de páginas para projetos grandes).

4. Escrevendo consultas

`queries.json`:

```
{
  "PSC": {
    "title": "Perovskite solar cell degradation under humidity",
    "note": "grounding block -- expect thousands",
    "groups": [
      ["perovskite solar cell", "halide perovskite photovoltaic"],
      ["degradation", "stability"],
    ]
  }
}
```

```

        ["humidity", "moisture"]]
    },
    "NOV": {
        "title": "novelty check: origami metamaterials as topological acoustic pumps",
        "note": "a SMALL number is the good outcome; read every hit",
        "groups": [["origami", "kirigami"], ["acoustic metamaterial", "phononic crystal"],
                    ["topological pumping", "edge state"]],
        "arxiv_groups": [0, 2]
    }
}

```

Regras práticas:

- Não coloque termos entre aspas; a ferramenta faz isso para a gramática de cada base.
- Uma palavra genérica sozinha (`model`, `structure`, `system`) em seu próprio grupo é a causa habitual de contagens nas dezenas de milhares.
- `arxiv_groups` indica quais grupos (no máximo dois) o arXiv recebe; o arXiv trava com booleanos profundamente aninhados. Padrão: os dois primeiros. O arXiv é paginado 100 registros por vez com pausa de 3 s, então um `--limit` grande é lento ali.
- O bloco mais informativo é uma interseção deliberada de duas literaturas que você suspeita não conversarem entre si. Um resultado próximo de zero é um achado — *se* você então ler cada resultado.
- Operadores de proximidade (`NEAR/n`, `W/n`) não são expressáveis; se o seu artigo precisa deles, mantenha ao lado strings escritas à mão para a Web of Science / Scopus e cite essas.

5. Executando uma busca

```

python librarian.py                # all blocks, every configured backend
python librarian.py --counts-only  # hit counts only (seconds)
python librarian.py --blocks NOV CD # selected blocks
python librarian.py --backends openalex ads
python librarian.py --skip arxiv   # drop a misbehaving backend
python librarian.py --limit 500    # records per block and backend (default 300)
python librarian.py --pdfs --pdf-blocks NOV # legal OA-PDF links via Unpaywall
python librarian.py --keep-junk     # keep non-curated venues (Zenodo, SSRN...)
python librarian.py --outdir lit_topomat # another research directory
python librarian.py --report-level intermediate --report-format md html pdf
python librarian.py --report-lang pt-BR # report in Portuguese (en, pt-BR, es, de, fr; §7.8)
python librarian.py --no-report
python librarian.py --queries other.json # another query file (default ./queries.json)
python librarian.py --backends-file b.json # another backends config; --init-backends writes the default
python librarian.py --timeout 60          # per-request timeout in seconds (default 45)
python librarian.py --list                # blocks and backend readiness, then exit
python librarian.py --selftest            # ping every backend, then exit

```

Lista completa de parâmetros: `python librarian.py --help`. Toda opção tem um padrão; `--outdir`, `--verbose`, `--quiet` e `--log-dir` existem em todos os scripts.

O que uma rodada grava (`lit/runs/<stamp>/`):

Arquivo	Conteúdo
<code>counts.json</code> , <code>counts.md</code>	contagens de resultados por bloco e backend; tabela pronta para colar
<code>queries.json</code>	a string de consulta exata enviada a cada backend
<code>blocks.json</code>	as definições de bloco usadas
<code>meta.json</code>	configurações, backends e endpoints, versão, tempos
<code>records/<block>_<backend>.json</code>	registros brutos por backend (após o filtro de veículos)

Arquivo	Conteúdo
<code>ris/<block>_<backend>.ris</code>	RIS por bloco para Zotero/Mendeley/EndNote
<code>all_records.json/.csv/.ris</code>	deduplicados, ordenados por citações
<code>all_records.bib, all_records.csl.json</code>	o mesmo conjunto como BibTeX e CSL-JSON
<code>junk.json</code>	registros removidos pelo filtro de veículos, com seus veículos
<code>prisma.json</code>	modelo para as etapas manuais do PRISMA
<code>run.log</code>	tudo o que foi impresso
<code>report.*</code>	o relatório (veja §7)

Mais `lit/counts_history.csv` (uma linha por bloco/backend/rodada, para a deriva) e `lit/logs/librarian_<stamp>_<pid>.1` (log de auditoria: invocação, versões, cada mensagem).

As contagens são salvas em checkpoint após cada chamada de API e Ctrl-C é seguro: um travamento no fim de uma rodada longa não perde nada.

6. O diretório de pesquisa

6.1 Índice

```
python project.py init --name "Topological materials review" --description "..."  
python project.py status
```

`status` lista cada membro (rodadas e fontes manuais) com data, número de registros, método e rótulo, o estado da pasta de entrada e o último relatório. Os membros são descobertos listando o diretório — nada precisa ser declarado. `project.json` guarda apenas o que não pode ser descoberto:

```
{"name": "...", "description": "...", "created": "2026-08-28",  
  "exclude": ["20260814T223331"],  
  "labels": {"20260828T095041": "August full scan"},  
  "block_aliases": {"X": "CD"},  
  "defaults": {"level": "simple", "format": ["md"], "metric": "openalex_2yr"}}
```

```
python project.py exclude 20260814T223331      # a test run you do not want in reports  
python project.py label 20260828T095041 "August full scan"  
python project.py alias X CD                   # block renamed between runs  
python project.py oa                           # Unpaywall pass over every member that lacks OA data  
python project.py oa --members 20260828T095041 # restrict the pass to these member ids
```

`oa` é a consulta de acesso aberto a posteriori: rodadas feitas sem `--pdfs` e fontes manuais recebem os campos `is_oa` / `oa_pdf` (só cópias legais, em cache em `unpaywall_cache.json`), que as estatísticas de acesso aberto do relatório e `--oa-only` passam então a cobrir para o projeto inteiro.

6.2 Trazendo registros de fora

Três maneiras, todas terminando em `lit/manual/<name>/` com o arquivo original, um `records.json` no esquema comum e um `source.json` com a proveniência:

1. **Linha de comando** — a maneira totalmente descrita:

```
python project.py ingest export.ris --name zotero-aug --block CD \  
  --method citation --who "A. Colleague" --origin "Zotero group library" \  
  --note "reference lists of the three key papers"
```

Vários arquivos podem ser dados; `--kind` sobrepõe a detecção por extensão (`ris`, `bibtex`, `csv`, `json`).

2. **Pasta de entrada** — solte arquivos em `lit/inbox/` e rode `python project.py ingest --inbox`; cada arquivo vira uma fonte com o seu nome (acrescente `--method` etc. para aplicar a todos).

3. **Web of Science** — `python wos_manual.py ingest` lê os arquivos RIS que você exportou da interface da WoS e os registra como fontes manuais com `method=database`.

Formatos aceitos: RIS (Zotero, Mendeley, EndNote, Web of Science, Scopus), BibTeX, CSV com linha de cabeçalho (nomes de coluna da Scopus e da WoS reconhecidos; senão `title`, `year`, `doi`, `journal`, `authors`, `url`, `abstract`, `block`, `cited_by`) e listas de registros JSON (por exemplo o `all_records.json` da rodada de um colega).

--method segue as categorias do PRISMA 2020 para registros identificados por outros métodos: `database` (uma exportação de base — entra na coluna das bases), `citation` (listas de referências, artigos citantes), `website`, `organisation`, `expert` (a recomendação de um colega), `other`.

Você também pode entregar arquivos extras a um único relatório sem armazená-los: `report.py --records file.ris`.

6.3 Do Zotero, Mendeley e EndNote

Saída: cada rodada grava RIS (`all_records.ris`, `ris/` por bloco), BibTeX (`all_records.bib`) e CSL-JSON (`all_records.csl.json`); importe com File → Import. Resumos, DOIs e URLs são levados, e o nome do bloco chega como palavra-chave (`block:NOV`), de modo que os itens importados já vêm marcados.

Entrada: exporte uma coleção como RIS (Zotero: botão direito → Export Collection → RIS; Mendeley: File → Export → RIS; EndNoteX: File → Export → RefMan RIS) e ingira como acima. Não há conexão ao vivo com a API do Zotero (roteiro).

7. Relatórios

7.1 Uma rodada

```
python report.py lit/runs/20260828T095041
python report.py --latest --level full --format html pdf
```

7.2 O diretório de pesquisa inteiro

```
python report.py --project
python report.py --project --outdir lit_topomat --level intermediate --format md html
```

Os relatórios vão para `lit/reports/<stamp>-<level>/`. O relatório de projeto acrescenta uma tabela de **Fontes** (cada rodada e fonte manual, sua data, método, registros e “novos aqui” — os registros únicos que nenhuma fonte anterior havia encontrado), uma **Linha do tempo** (contagens por bloco ao longo das rodadas; quando os registros entraram no projeto), um fluxo PRISMA com as duas colunas de identificação e, quando `journals.py` foi executado, métricas de periódicos.

7.3 Níveis

Nível	Seções
simple	metadados; fontes; estratégia de busca com a string exata por backend; resumo dos resultados; linha do tempo; fluxo PRISMA 2020 + checklist PRISMA-S; 10 principais registros por bloco; sugestões
intermediate	+ cada registro único; sobreposição de fontes (“encontrado só aqui”); distribuições por ano / veículo / autor; métricas de periódicos; veículos filtrados; erros; estatísticas de acesso aberto; histórico de contagens

Nível	Seções
full	+ cada registro com resumo completo, lista de autores e quais fontes o encontraram; listas brutas por fonte antes da deduplicação; os registros filtrados; configuração dos backends; arquivos project.json e source.json; o log da rodada; ambiente

Tamanhos, a partir do exemplo distribuído (quatro blocos, três bases CC0, 1.226 registros únicos): 6, 68 e 427 páginas de PDF.

7.4 Formatos

md (Markdown; renderiza no GitHub), html (autocontido, claro/escuro, imprimível, diagrama SVG), tex (LaTeX com diagrama TikZ), pdf, txt (texto puro, diagrama ASCII). O PDF é compilado do LaTeX com xelatex, lualatex ou pdflatex se um deles estiver instalado, senão com o pandoc, senão por um escritor embutido que diagrama a versão em texto — a opção nunca falha.

7.5 Filtros

Opção	Efeito
--since DATE, --until DATE	mantém os membros (rodadas / fontes manuais) buscados na janela
--latest	só o membro mais recente (projeto); a rodada mais nova (modo simples)
--diff	mantém só os registros <i>vistos pela primeira vez</i> dentro da janela — “o que as buscas desde DATE acrescentaram”
--year-from Y, --year-to Y	ano de publicação
--backends a b	bases / fontes a incluir (fontes manuais são manual:<name>)
--blocks A CD	blocos a incluir
--sources auto\ manual\ all	tipos de membro
--records FILE...	RIS/BibTeX/CSV/JSON extras como fonte manual transitória
--metric NAME --min-metric X	mantém registros cuja métrica de veículo é pelo menos X (veja §8)
--min-citations N	limiar de citações
--oa-only	só registros com cópia legal de acesso aberto (precisa de dados de --pdfs ou project.py oa)
--top N, --sort cited\ year\ metric	tamanho e ordem da tabela
--basename, --out	radical do nome de arquivo e diretório de saída

Os filtros são listados na tabela de metadados do relatório e no item 9 do PRISMA-S, para que um relatório filtrado nunca seja confundido com a busca inteira.

7.6 PRISMA

O relatório traz um diagrama de fluxo PRISMA 2020 (SVG no HTML, TikZ no LaTeX/PDF, ASCII no Markdown e no texto) e um checklist de relato de busca PRISMA-S. A ferramenta preenche o que pode saber: registros identificados por base (somados sobre as rodadas no modo projeto), registros identificados por outros métodos (fontes manuais por método), registros recuperados, removidos por automação (o filtro de veículos), duplicatas removidas, registros restantes para triagem. Ela é explícita sobre a diferença entre *identificados* (o que cada base informa) e *recuperados* (o que foi baixado dentro de --limit).

As etapas que só um humano pode conhecer são lidas de `prisma.json` (rodada única) ou `screening.json` (diretório de pesquisa); um modelo com valores `null` é gravado no primeiro relatório. Preencha os inteiros conforme a triagem avança e rode o relatório de novo:

```
{"records_screened": 2216, "records_excluded": 2100,
 "reports_sought": 116, "reports_not_retrieved": 4, "reports_assessed": 112,
 "excluded_reasons": {"not_topological": 60, "no_experiment": 30},
 "other_sought": 12, "other_not_retrieved": 0, "other_assessed": 12,
 "other_excluded_reasons": {"duplicate_of_included": 3},
 "studies_included": 22, "reports_included": 31,
 "citation_searching": "reference lists of the 22 included studies",
 "prior_work": "none", "peer_review": "search strategy reviewed by the librarian"}
```

7.7 Sugestões

Baseadas em regras, no fim de cada relatório: chamadas de backend que falharam, blocos com milhares de resultados, blocos de tamanho de novidade (leia cada resultado), limite `--limit` atingido, uma base com alta proporção de veículos filtrados, nenhum backend com grau de citação, consulta de acesso aberto não executada, etapas PRISMA não preenchidas, sem métricas de periódicos, deriva das contagens entre rodadas e — no modo projeto — a ausência de qualquer fonte manual.

7.8 Idiomas

```
python report.py --latest --lang pt-BR
python report.py --project --lang de --format pdf
python librarian.py --report-lang fr # the report written at the end of a run
```

`--lang` (`report.py`) e `--report-lang` (`librarian.py`) aceitam em (padrão), `pt-BR`, `es`, `de` ou `fr`; um diretório de pesquisa pode fixar o próprio padrão com `"defaults": {"lang": "es"}` em `project.json`, e uma opção explícita prevalece sobre ele. Só o texto próprio do relatório muda — títulos, cabeçalhos de tabela, as etapas PRISMA 2020 e o diagrama de fluxo em todos os formatos, o checklist PRISMA-S, os parágrafos explicativos e as sugestões — junto com o separador de milhar do idioma. O que a ferramenta encontrou ou recebeu é reproduzido exatamente como está, seja qual for o idioma: títulos, resumos, autores e veículos dos registros, os nomes e notas dos seus blocos, as strings de consulta exatas, os nomes dos backends, as opções citadas no texto, os nomes de arquivos, os despejos JSON e o log da rodada embutido. A saída no console, o `run.log` e os logs de auditoria são sempre em inglês, de modo que rodadas feitas em idiomas diferentes continuam pesquisáveis juntas.

8. Métricas de periódicos

```
python journals.py fetch # every journal seen in the directory
python journals.py fetch --providers openalex --refresh
python journals.py import-scimago scimagojr_2024.csv --year 2024 [--all]
python journals.py import-jcr JCR_JournalResults_*.csv # Journal Citation Reports downloads
python journals.py import-csv other.csv --provider my_metric --year 2023 --name-col Journal --
value-col Value [--issn-col ISSN] [--delimiter ";"] # any name/value table; ISSN
python journals.py list --missing jcr_if # journals still to look up by hand
python journals.py show --metric scopus_citescore
```

Armazém: `lit/journals/metrics.json`, uma entrada por periódico chaveada por ISSN (senão pelo nome normalizado), valores mantidos **por ano e nunca sobrescritos** — busque de novo no ano que vem e o relatório mostra a série.

Provedor	Chave	Métricas	Histórico
openalex	nenhuma	openalex_2yr (citação média em 2 anos, um número do tipo fator de impacto), openalex_h, obras/citações por ano	instantâneo sob o ano da busca
scopus	SCOPUS_API_KEY	scopus_citescore, sjr, snip	histórico completo por ano
scimago	nenhuma; baixe o CSV do ano em scimagojr.com	sjr, scimago_h, quartil	um arquivo por ano
jcr	licença	jcr_if	somente importação

O Journal Impact Factor (Clarivate JCR) é proprietário: não há API gratuita e a ferramenta não vai raspá-lo. Usuários licenciados baixam CSVs da página *Browse journals* do JCR (600 linhas por download; fatie por categoria e depois por quartil) e os importam com `journals.py import-jcr FILE...` — as colunas e o ano do JIF são detectados. `journals.py list --missing jcr_if` imprime os periódicos do seu diretório ainda sem valor, que é a lista a consultar. O protocolo completo está em `docs/JCR_IMPORT.md`. Para uma métrica que cubra todos os periódicos, o CSV do SCImago (~30.000 periódicos, um download) é o caminho prático; `--all` importa o arquivo inteiro, o padrão importa só os periódicos vistos nos seus registros.

Nos relatórios: uma coluna de métrica nas tabelas de registros, “veículos neste conjunto por métrica”, uma tabela de evolução para veículos com dois ou mais anos registrados, e o filtro `--min-metric`. `--metric` escolhe qual (padrão `openalex_2yr`, ou `defaults.metric` em `project.json`).

9. Web of Science

A gramática completa `TS=/NEAR` está na Expanded API, raramente licenciada; o nível gratuito Starter rejeita booleanos complexos. A Web of Science é, portanto, trabalho manual, tornado pequeno:

```
python wos_manual.py prep      # query files + CHECKLIST.md in WoS grammar
python wos_manual.py walk      # copies each query to the clipboard in turn
python wos_manual.py ingest    # RIS exports -> records, registered as manual sources
python wos_manual.py status
python wos_manual.py prep --queries other.json # a different query file (default ./queries.json)
```

O checklist codifica as configurações da interface que quebram consultas silenciosamente (Core Collection, busca Advanced, edições, forma marcada versus nua).

10. Logs e auditoria

Cada script grava `<outdir>/logs/<script>_<stamp>_<pid>.log` com a invocação exata, as versões da ferramenta e do Python, o diretório de pesquisa, cada aviso e erro, e o desfecho. A saída no console é pequena por padrão; `--verbose` mostra tudo, `--quiet` só avisos e erros; `--log-dir` move os logs. As rodadas mantêm adicionalmente `run.log` (a transcrição do console) no diretório da rodada.

11. Fluxos de trabalho

Uma verificação de novidade (uma tarde). Escreva 1–3 blocos de consulta cruzada; `--counts-only`; aperte tudo o que estiver nos milhares; rodada completa com `--pdfs`; leia as Sugestões; leia cada resultado dos blocos pequenos à mão; registre o que triou em `prisma.json`; rode `report.py` de novo; importe o RIS no Zotero.

Uma busca sistemática em um projeto (meses). `project.py init`. Repita o `librarian.py` em intervalos com o mesmo `queries.json`. Ingira as sessões da Web of Science e as exportações dos colegas. `report.py --project` para o quadro geral; `--project --since <último relatório> --diff` para o que é novo; `journals.py`

`fetch` anualmente. Preencha `screening.json` conforme avança; o diagrama PRISMA se completa sozinho, pronto para o material suplementar.

Um laboratório. Um diretório de pesquisa por projeto (`--outdir`); cada um tem seu próprio índice, triagem e relatórios. Pastas de entrada permitem que colaboradores soltem exportações sem aprender a ferramenta. Deliberadamente não há mesclagem entre projetos: perguntas diferentes, blocos diferentes.

Um exemplo trabalhado. `docs/WALKTHROUGH.md` (em inglês) conduz um projeto real de `queries.json` até um diagrama PRISMA completo, com todos os comandos.

Com um agente de IA. Aponte-o para `AGENTS.md`; peça que esboce o `queries.json` a partir da sua pergunta de pesquisa, rode as varreduras e percorra o relatório com você. O arquivo de consulta estruturado, os arquivos JSON e o relatório foram projetados para serem escritos e auditados por um agente.

12. Recursos e limitações

Recursos: uma consulta estrutural traduzida para nove gramáticas nativas; bases como configuração JSON (`--init-backends`); rodadas arquivadas e citáveis com strings de consulta exatas e histórico de contagens; checkpoints e Ctrl-C seguro; um filtro de veículos com comprovantes; seis backends sem chave; NASA ADS e INSPIRE para física; links legais de PDF de acesso aberto via Unpaywall; relatórios em três níveis e cinco formatos com PRISMA 2020 e PRISMA-S; diretórios de pesquisa com fontes manuais, proveniência, linha do tempo e relatórios diferenciais; métricas de periódicos com série por ano; logs de auditoria; uma suíte de testes offline (325 verificações) e CI.

Limitações, todas por projeto ou pelo mundo:

- As contagens não são comparáveis entre bases; operadores de proximidade são descartados. Descubra aqui; cite uma base no artigo.
- Os resultados da Scopus exigem direito institucional (rede/VPN). A API da Web of Science raramente é licenciada; use o caminho manual.
- O arXiv recebe no máximo dois grupos por bloco.
- `--limit` limita os registros por bloco e backend (mais citados primeiro); blocos grandes são uma fatia, não o conjunto completo. Aumente-o quando precisar de completude.
- O OpenAlex indexa repositórios não curados (~15 % dos seus registros); filtrados por padrão, mantidos em `junk.json`.
- Sem download de PDFs (só links do Unpaywall), sem snowballing, sem grafo de citações, sem conexão ao vivo com Zotero/Mendeley (roteiro); BibTeX e CSL-JSON são escritos, não lidos de volta de uma biblioteca Zotero.
- Métricas de periódicos: os valores do OpenAlex são instantâneos; o Fator de Impacto do JCR é proprietário e só importável; casar periódicos pelo nome é imperfeito quando um registro não tem ISSN.
- A deduplicação é por DOI, senão pelos primeiros 90 caracteres do título; pares preprint/publicado com títulos diferentes sobrevivem como dois registros.
- O Google Scholar não é e não será um backend (sem API; raspar viola seus termos).

13. Testes

```
python tests/test_librarian.py
```

Offline, só biblioteca padrão, sem chaves: os backends rodam contra respostas de API gravadas, o gerador de relatórios contra rodadas e diretórios de pesquisa sintéticos, e a linha de comando de cada script é exercitada de ponta a ponta. O arquivo também é um módulo `pytest` (`pytest tests/`). O CI roda o `pyflakes` e a suíte em Linux, Windows e macOS sob Python 3.9 e 3.13.

14. Licença e conduta

Apache License 2.0. A ferramenta é feita para tornar o respeito aos termos de serviço de cada base o caminho fácil: só APIs documentadas, limites de taxa honrados, um endereço de contato em cada requisição, sem raspagem, sem

contorno de paywall.